

算法设计与分析

Analysis and Design of Algorithm

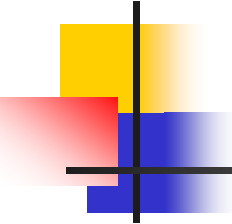
Lesson 13

要点回顾

■ 回溯法

- 一种组织得井井有条的(带有系统性)、能避免不必要搜索的(带有跳跃性)穷举式搜索法

问题	解性质	解向量	搜索空间	搜索方式	约束条件
n 后问题	可行解	$\langle x_1, x_2, \dots, x_n \rangle$ x_i :第 i 行列号	n 叉树	深度优先	彼此不攻击
0-1背包问题	最优解	$\langle x_1, x_2, \dots, x_n \rangle$ $x_i=0/1$ $x_i=1 \Leftrightarrow$ 选 i	子集树	深度优先	不超过背包重量
TSP问题	最优解	$\langle k_1, k_2, \dots, k_n \rangle$ $1, 2, \dots, n$ 的排列	排列树	深度优先	选未经过的城市



要点回顾(cont.)

- 回溯法适用条件

- 多米诺性质

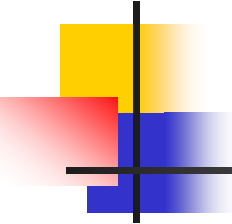
$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

- 逆否命题:

$$\neg P(x_1, x_2, \dots, x_k) \rightarrow \neg P(x_1, x_2, \dots, x_{k+1}) \quad 0 < k < n$$

- 作用:

k 维向量如果不满足约束条件, 扩张向量到 $k+1$ 维仍旧不满足, 可以回溯!



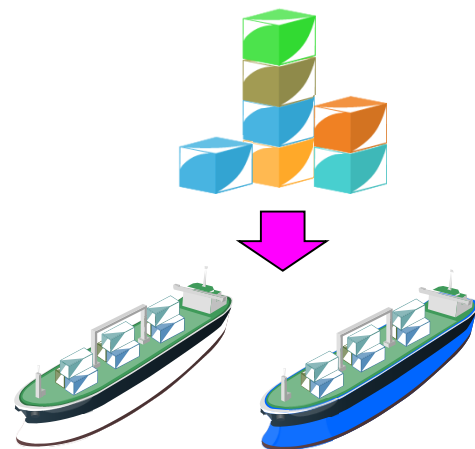
要点回顾(cont.)

■ 回溯法设计要素

- **适用对象：** 求解搜索问题和优化问题
- **搜索空间：** 树，结点对应部分解向量，可行解在树叶上
- **搜索过程：** 采用系统的方法隐含遍历搜索树
- **搜索策略：** 深度/宽度优先、函数优先、宽深结合
- **结点分支判定条件：**
 - 满足约束条件——分支扩张解向量
 - 不满足约束条件——回溯到该结点的父结点
- **结点状态：** 动态生成
 - 可以约定(黑：已访问；灰：正在访问；白：未访问)
- **存储：** 当前路径

装载问题

问题： 有 n 个集装箱，需要装上两艘载重分别为 c_1 和 c_2 的轮船。 w_i 为第 i 个集装箱的重量，且 $w_1 + w_2 + \dots + w_n \leq c_1 + c_2$ 。问是否存在一种合理的装载方案把这 n 个集装箱装上船？如果有，给出一种方案。



装载问题

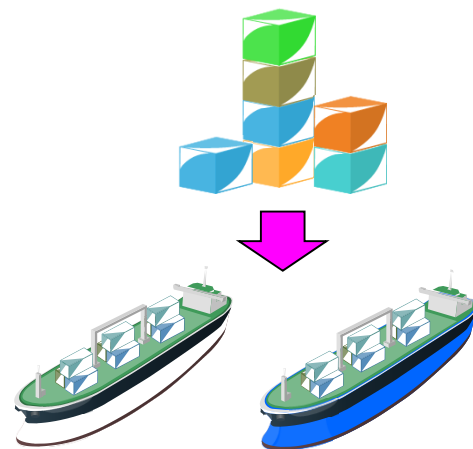
问题： 有 n 个集装箱，需要装上两艘载重分别为 c_1 和 c_2 的轮船。 w_i 为第 i 个集装箱的重量，且 $w_1+w_2+\dots+w_n \leq c_1+c_2$ 。问是否存在一种合理的装载方案把这 n 个集装箱装上船？如果有，给出一种方案。

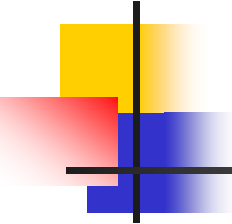
实例：

$W = \langle 90, 80, 40, 30, 20, 12, 10 \rangle$

$c_1 = 152, c_2 = 130$

解： 1,3,6,7装第1艘船，其余第2艘





求解思路

输入： $W = \langle w_1, w_2, \dots, w_n \rangle$ 为集装箱重量， c_1 和 c_2 为船的最大载重

算法思想： 令第一艘船的载入量为 W_1

1. 用回溯法求使得 $c_1 - W_1$ 达到最小的装载方案
2. 若满足

$$w_1 + w_2 + \dots + w_n - W_1 \leq c_2$$

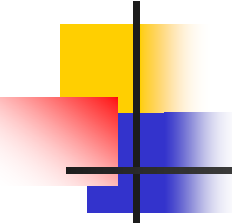
则回答 “YES”，否则回答 “NO”

算法伪码

■ 算法Loading(W, c_1)

1. Sort(W)
2. $B \leftarrow c_1; best \leftarrow c_1; i \leftarrow 1$
3. while $i \leq n$ do
4. if 装入 i 后重量不超过 c_1 then
5. $B \leftarrow B - w_i; x[i] \leftarrow 1; i \leftarrow i + 1$
6. else $x[i] \leftarrow 0; i \leftarrow i + 1$
7. if $B < best$ then 记录解; $best \leftarrow B$
8. Backtrack(i) $\longrightarrow$ 回溯
9. if $i = 1$ then return 最优解
10. else goto 3

B 为当前空隙
 $best$ 为最小空隙



子过程 Backtrack

■ 算法 Backtrack(i)

1. while $i > 1$ and $x[i]=0$ do
2. $i \leftarrow i-1$
3. if $x[i]=1$
4. then $x[i] \leftarrow 0$
5. $B \leftarrow B + w_i$
6. $i \leftarrow i+1$

以上是迭代实现过程，递归实现如何做？

算法伪码

■ 算法Loading(W, c_1)

1. Sort(W)
2. $B \leftarrow c_1$; $best \leftarrow c_1$; $i \leftarrow 1$
3. while $i \leq n$ do
4. if 装入 i 后重量不超过 c_1 then
5. $B \leftarrow B - w_i$; $x[i] \leftarrow 1$; $i \leftarrow i + 1$
6. else $x[i] \leftarrow 0$; $i \leftarrow i + 1$
7. if $B < best$ then 记录解; $best \leftarrow B$
8. Backtrack(i) $\longrightarrow$ 回溯
9. if $i = 1$ then return 最优解
10. else goto 3

■ 算法Backtrack(i)

1. while $i > 1$ and $x[i] = 0$ do
2. $i \leftarrow i - 1$
3. if $x[i] = 1$
4. then $x[i] \leftarrow 0$
5. $B \leftarrow B + w_i$
6. $i \leftarrow i + 1$

实例

$W = \langle 90, 80, 40, 30, 20, 12, 10 \rangle$

$c_1 = 152, c_2 = 130$

解：

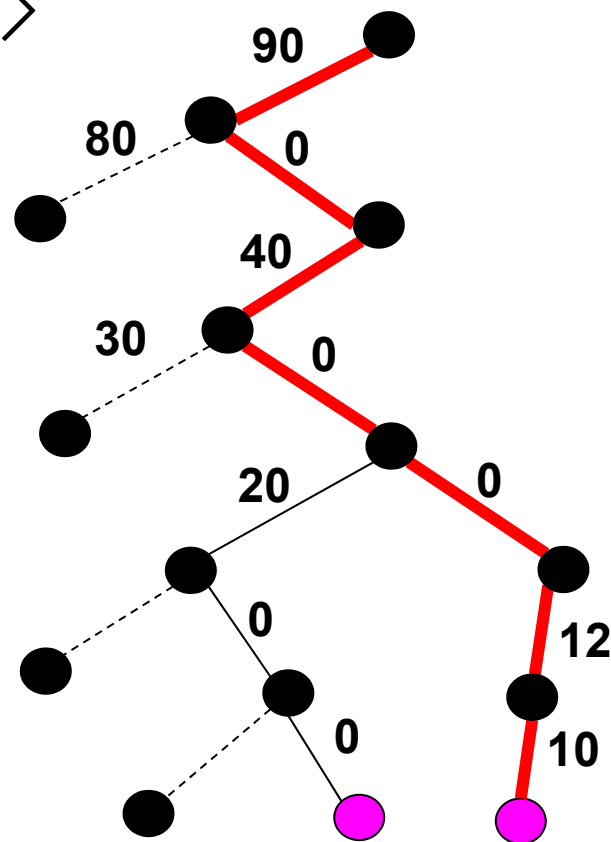
可以装！

方案如下：

1,3,6,7装第一艘船

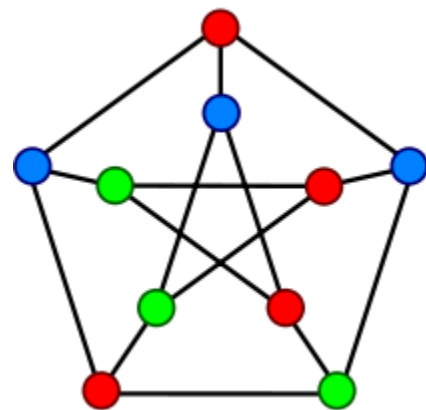
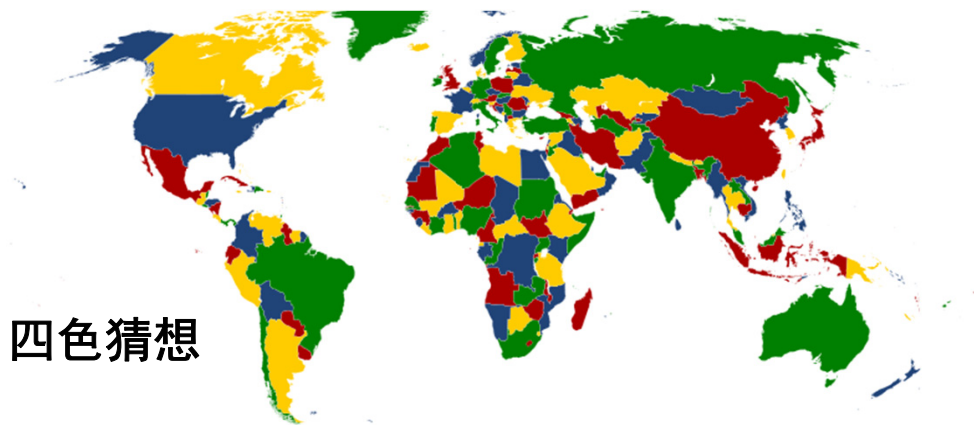
2,4,5装第二艘船

时间复杂度： $O(2^n)$

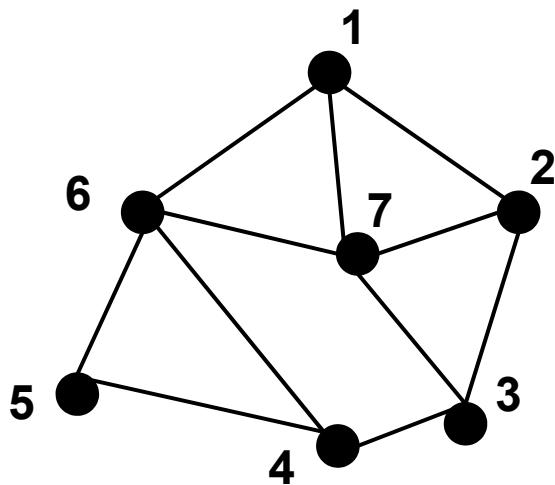


图的着色问题

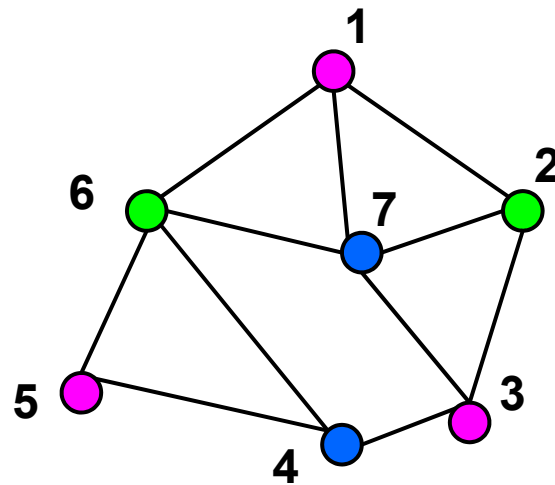
- **输入：** 无向连通图 G 和 m 种颜色的集合，用这些颜色给图的顶点着色，每个顶点一种颜色。要求是： G 的每条边的两个顶点着不同颜色。
- **输出：** 所有可能的着色方案。如果不存在着色方案，回答“NO”

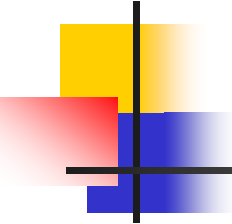


实例



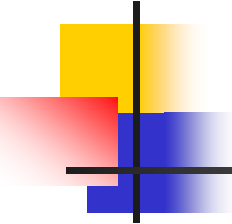
$$n=7, m=3$$





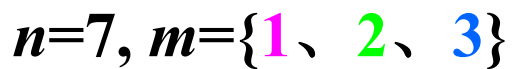
解向量

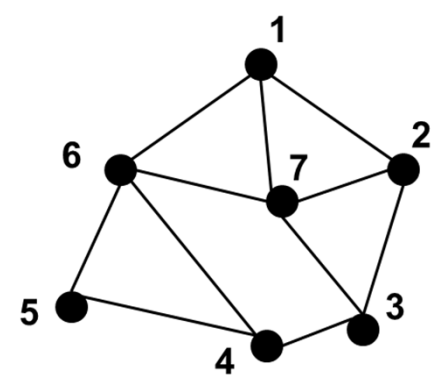
- 设 $G=(V, E)$, $V=\{1,2,\dots,n\}$
- 颜色编号: $1,2, \dots, m$
- **解向量:** $\langle x_1, x_2, \dots, x_n \rangle$
 - $x_i \in \{1,2,\dots,m\}$
- 结点的部分向量 $\langle x_1, x_2, \dots, x_k \rangle$, $1 \leq k \leq n$
 - 表示只给顶点 $1,2,\dots,k$ 着色的部分方案



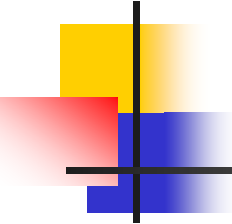
算法设计

- **搜索树：** m 叉树
- **约束条件：** 在结点 $\langle x_1, \dots, x_k \rangle$ 处，顶点 $k+1$ 的邻接表中结点已用过的颜色不能再用。如果邻接表中结点已用过 m 种颜色，则结点 $k+1$ 没法着色，从该结点回溯到其父节点。→ 满足多米诺性质
- **搜索策略：** 深度优先
- **时间复杂度：** $O(n \times m^n)$



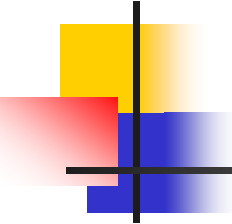


第一个解向量: $\langle 1, 2, 1, 3, 1, 2, 3 \rangle$



时间复杂度与改进途径

- 时间复杂度： $O(n \times m^n)$
- 根据**对称性**，只需搜索1/3的解空间。当1和2确定，即<1,2>以后，只有1个解，在<1,3>为根的子树中，也只有一个解。由于3个子树的对称性，总共有6个解。
- 在取定<1,2>后，不可扩张成<1,2,3>，因为7和1,2,3都相邻。7没法着色。可以从**打叉的结点**回溯，而不必搜索其子树。



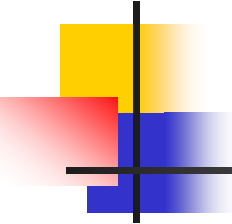
着色问题的应用

- 会场分配问题：

- 有 n 项活动需要安排，对于活动 i, j ，如果 i, j 时间冲突，就说 i, j 不相容。如何分配这些活动，使得每个会场的活动相容且占用会场数最少？

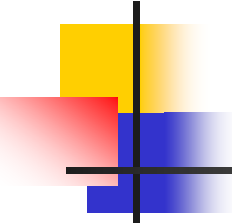
- 建模：

- 活动作为图的顶点，如果 i, j 不相容，则在 i 和 j 之间加一条边，会场标号作为颜色标号。求图的一种着色方案，使得使用的颜色数最少。



第五章小结

- 理解回溯法的深度优先搜索策略
- 掌握用回溯法解题的算法框架
 - 递归回溯最优子结构性质
 - 迭代回溯贪心选择性质
 - 子集树算法框架
 - 排列树算法框架
- 通过应用范例学习回溯法的设计策略
 - n 后问题、0-1背包问题、旅行售货员问题
 - 装载问题
 - 图的着色问题



课程内容

NP完全性理论与近似算法

算法高级理论

随机化算法

线性规划与网络流

高级算法

递归
分治

动态
规划

贪心
算法

回溯与
分支限界

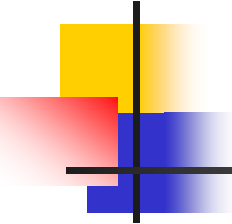
基础算法

算法分析与问题的计算复杂性

算法基础理论

第六章 分支限界法

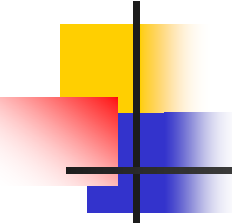
Branch & Bound



学习要点

- **理解**分支限界法的剪枝搜索策略
- **掌握**分支限界法的算法框架
 - 队列式分支限界法
 - 优先队列式分支限界法
- **通过应用范例学习**分支限界法的设计策略
 - 背包问题、0-1背包问题
 - 非对称旅行商问题
 - 单源最短路径问题
 - 装载问题
 - 最大团问题

分支限界法概述



分支限界法与回溯法的区别

■ 求解目标：

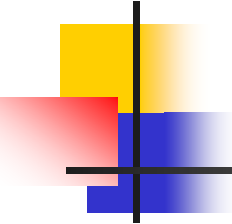
- 一般情况下，回溯法求解目标是找出解空间树中满足约束条件的所有解，而分支限界法求解目标则是找出满足约束条件的一个解，或是在满足约束条件的解中找出在某种意义下的最优解（**可以理解为更细粒度的改进型回溯法**）。

■ 搜索策略：

- 回溯法更多地以深度优先的方式搜索解空间树，而分支限界法则更多地以宽度优先结合最小耗费优先（**函数优先**）的方式搜索解空间树。

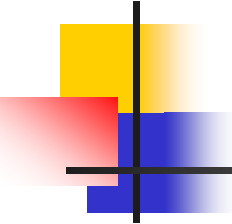
■ 各自特点：

- 回溯法空间效率高；分支限界法往往更**快**。



分支限界法的基本思想

- 分支限界法常以**宽度优先**结合**最小耗费（最大效益）**优先的方式搜索问题的解空间树。
- 对已处理的各结点根据**限界函数**估算目标函数的可能取值，从中选取使目标函数取得极值（极大/极小）的结点优先进行宽度优先搜索→不断调整搜索方向，尽快找到解。
- **特点：**限界函数常基于问题的目标函数，适用于求解最优化问题。



常见的分支限界法

- 队列式分支限界法
 - 按照队列**先进先出**原则选取下一个结点为扩展结点。
- 优先队列式分支限界法
 - 按照规定的**结点费用最小**原则选取下一个结点为扩展结点（常采用优先队列实现）。
- 栈式分支限界法
 - 按照栈**后进先出**原则选取下一个结点为扩展结点。

组合优化问题的分支限界法

组合优化问题

- 组合优化问题的相关概念
 - 目标函数（极大化或极小化）
 - 约束条件（解满足的条件）
 - 可行解：搜索空间满足约束条件的解
 - 最优解：使得目标函数达到极大（极小）的可行解

- 一般背包问题

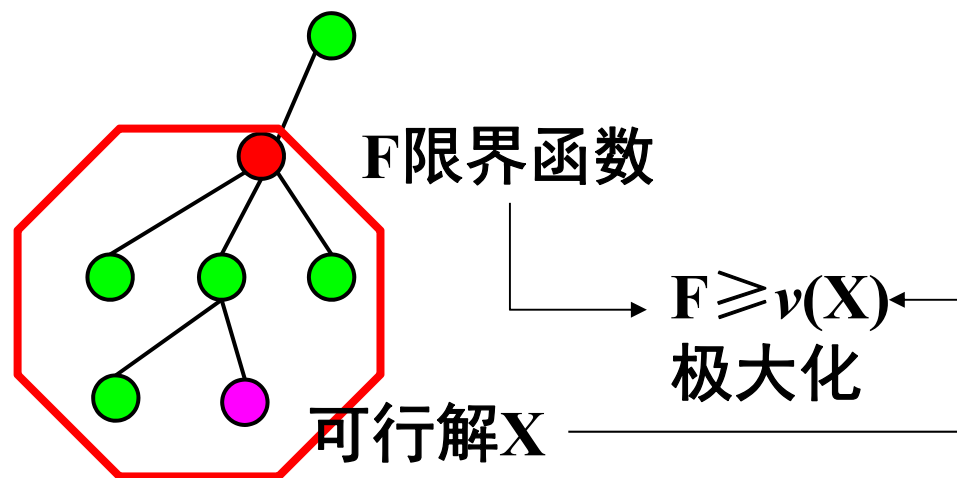
$$\max (x_1 + 3x_2 + 5x_3 + 9x_4)$$

$$s.t. \quad \begin{cases} 2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10 \\ x_i \in N, i = 1, 2, 3, 4 \end{cases}$$



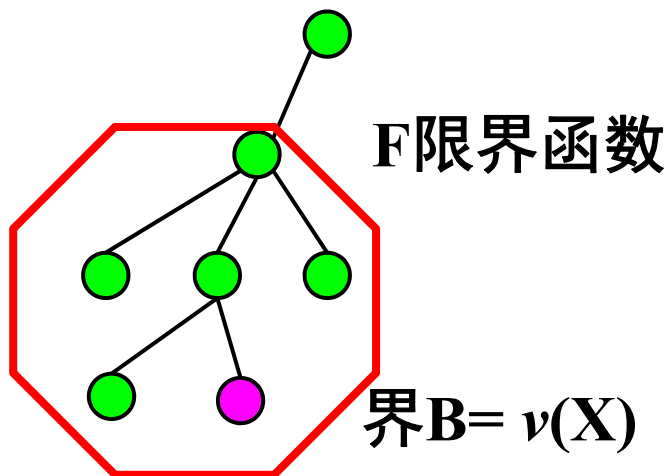
限界函数

- **计算位置：** 搜索树的结点
- **估值：** 极大化问题是以该点为根的子树所有可行解的值的**上界**（极小化问题则为下界）
- **性质：** 对极大化问题父节点代价**不小于**子结点的代价（极小化问题则相反）

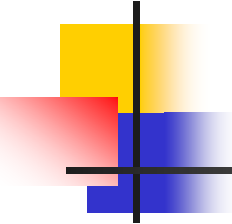


界

- **含义：**当前得到可行解的**目标函数**最大值（极小化问题则相反）
- **初值：**极大化问题初值为0（极小化问题则为最大值）
- **更新：**得到更好的可行解时



界 $\subseteq$ 限界函数，即： $F \geq B$



分支限界

- 停止分支，回溯父节点的依据
 1. 不满足约束条件
 2. 对于极大化问题，限界函数值小于当前界（对于极小化问题是大于界）
- 界的更新
 - 对极大化问题，如果一个新的可行解的优化函数值大于（极小化问题为小于）当前的界，则把界更新为该可行解的值

分支限界解组合优化问题

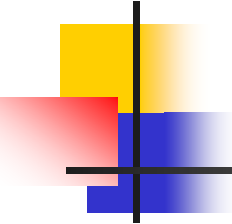
■ 背包问题

背包限重为10

物品 <i>i</i> 属性	1	2	3	4
价值 v_i	1	3	5	9
重量 w_i	2	3	4	7

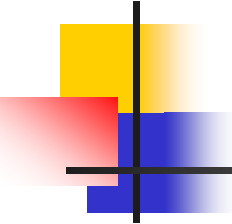
$$\max (x_1 + 3x_2 + 5x_3 + 9x_4)$$

$$s.t. \quad \begin{cases} 2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10 \\ x_i \in N, i = 1, 2, 3, 4 \end{cases}$$



限界函数的设定

- 对结点 $\langle x_1, x_2, \dots, x_k \rangle$, 估计以该结点为根的子树中可行解的上界
- 按单位重量的价值 v_i / w_i 从大到小排序
- 限界函数=已装入价值+ Δ
 - Δ : 还可以继续装入最大价值的上界
 - $\Delta = \text{背包剩余重量} \times v_{k+1} / w_{k+1}$ (可装)
 - $\Delta = 0$ (不可装)



实例：背包问题

$$\max (x_1 + 3x_2 + 5x_3 + 9x_4)$$

$$s.t. \quad \begin{cases} 2x_1 + 3x_2 + 4x_3 + 7x_4 \leq 10 \\ x_i \in N, i = 1, 2, 3, 4 \end{cases}$$

对**变元重新排序**使得 $\frac{v_i}{w_i} \geq \frac{v_{i+1}}{w_{i+1}}$

排序后

$$\max (9x_1 + 5x_2 + 3x_3 + x_4)$$

$$s.t. \quad \begin{cases} 7x_1 + 4x_2 + 3x_3 + 2x_4 \leq 10 \\ x_i \in N, i = 1, 2, 3, 4 \end{cases}$$